

Documentación API CRUD con Docker y CI/CD

Rafael Núñez Pedrosa

7 de diciembre de 2025

Índice

1. Introducción	3
2. Arquitectura del Sistema	3
2.1. Componentes principales	3
3. Workflow de GitHub Actions	3
3.1. Archivo docker-push.yml	3
3.1.1. Configuración de triggers	3
3.1.2. Variables de entorno	4
3.2. Jobs y Steps del Pipeline	4
3.2.1. Job: build-and-push	4
3.2.2. Step 1: Checkout del código	4
3.2.3. Step 2: Configuración de Docker Buildx	5
3.2.4. Step 3: Login en Docker Hub	5
3.2.5. Step 4: Construcción y publicación de la imagen	5
3.2.6. Step 5: Información de la imagen	6
3.3. Sistema de Notificaciones a Telegram	6
3.3.1. Notificación de éxito	6
3.3.2. Notificación de error	7
4. Configuración de Secrets	7
4.1. Secrets requeridos en GitHub	7
4.2. Cómo configurar los secrets	7
5. Flujo de Trabajo Completo	8
5.1. Proceso automatizado	8
6. Ventajas del Sistema	8
6.1. Automatización	8
6.2. Trazabilidad	8
6.3. Notificaciones en tiempo real	9
6.4. Optimización	9

1. Introducción

Este documento describe la implementación de un sistema de Integración Continua y Despliegue Continuo (CI/CD) para una API CRUD desarrollada con Node.js, Express y MongoDB. El sistema automatiza la construcción y publicación de imágenes Docker en Docker Hub mediante GitHub Actions, además de enviar notificaciones a Telegram sobre el estado del despliegue.

2. Arquitectura del Sistema

2.1. Componentes principales

- **API REST:** Aplicación Node.js con Express y Mongoose
- **Base de datos:** MongoDB
- **Contenedorización:** Docker y Docker Compose
- **CI/CD:** GitHub Actions
- **Notificaciones:** Bot de Telegram

3. Workflow de GitHub Actions

3.1. Archivo docker-push.yml

El archivo `.github/workflows/docker-push.yml` define el pipeline de CI/CD que automatiza el proceso de construcción y publicación de la imagen Docker.

3.1.1. Configuración de triggers

Listing 1: Configuración de eventos que disparan el workflow

```
name: Build and Push Docker Image

on:
  push:
    branches:
      - main
      - master
    paths:
      - 'package.json'
      - 'Dockerfile'
      - 'main.js'
      - 'docker-compose.yml'
      - '.github/workflows/docker-push.yml'
  workflow_dispatch:
```

El workflow se ejecuta automáticamente cuando:

- Se hace push a las ramas `main` o `master`
- Se modifican archivos específicos: `package.json`, `Dockerfile`, `main.js`, `docker-compose.yml` o el propio workflow
- Se ejecuta manualmente mediante `workflow_dispatch`

3.1.2. Variables de entorno

Listing 2: Variables de entorno del workflow

```
env:  
  DOCKER_USERNAME: rafadockerhub  
  DOCKER_IMAGE_NAME: rafadockerhub/api-crud-robert  
  DOCKER_TAG: latest
```

Estas variables definen:

- `DOCKER_USERNAME`: Usuario de Docker Hub
- `DOCKER_IMAGE_NAME`: Nombre completo de la imagen Docker
- `DOCKER_TAG`: Etiqueta de la versión (latest)

3.2. Jobs y Steps del Pipeline

3.2.1. Job: build-and-push

Listing 3: Configuración del job principal

```
jobs:  
  build-and-push:  
    runs-on: ubuntu-latest  
    environment: DOCKER
```

El job se ejecuta en un runner de Ubuntu y utiliza el entorno `DOCKER` para gestionar secrets.

3.2.2. Step 1: Checkout del código

Listing 4: Obtener el código del repositorio

```
- name: Checkout código  
  uses: actions/checkout@v4
```

Este step descarga el código fuente del repositorio en el runner.

3.2.3. Step 2: Configuración de Docker Buildx

Listing 5: Configurar Docker Buildx

```
- name: Configurar Docker Buildx
  uses: docker/setup-buildx-action@v3
```

Docker Buildx es una extensión de Docker CLI que permite construcciones avanzadas con caché y soporte multi-plataforma.

3.2.4. Step 3: Login en Docker Hub

Listing 6: Autenticación en Docker Hub

```
- name: Login a Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${ secrets.DOCKER_USERNAME }
    password: ${ secrets.DOCKER_PASSWORD }
```

Autentica el runner en Docker Hub utilizando credenciales almacenadas en GitHub Secrets:

- `secrets.DOCKER_USERNAME`: Usuario de Docker Hub
- `secrets.DOCKER_PASSWORD`: Token o contraseña de Docker Hub

3.2.5. Step 4: Construcción y publicación de la imagen

Listing 7: Build y push de la imagen Docker

```
- name: Construir y subir imagen Docker
  uses: docker/build-push-action@v5
  with:
    context: .
    file: ./Dockerfile
    push: true
    tags: ${ env.DOCKER_IMAGE_NAME }:${ env.DOCKER_TAG }
    cache-from: type=registry,ref=${ env.DOCKER_IMAGE_NAME
    }:buildcache
    cache-to: type=inline
```

Parámetros importantes:

- `context`: Directorio de contexto para la construcción
- `file`: Ubicación del Dockerfile
- `push`: Sube la imagen a Docker Hub automáticamente
- `tags`: Etiquetas aplicadas a la imagen
- `cache-from/cache-to`: Optimización mediante caché de capas

3.2.6. Step 5: Información de la imagen

Listing 8: Mostrar información de la imagen construida

```
- name: Mostrar informacion de la imagen
run: |
  echo "Imagen construida y subida exitosamente:"
  echo "  - Imagen: ${ env.DOCKER_IMAGE_NAME }}:${ env.DOCKER_TAG }"
  echo "  - Docker Hub: https://hub.docker.com/r/${ secrets.DOCKER_USERNAME }/api-crud-robert"
```

Muestra información relevante en los logs del workflow.

3.3. Sistema de Notificaciones a Telegram

3.3.1. Notificación de éxito

Listing 9: Notificación cuando el despliegue es exitoso

```
- name: Enviar notificacion a Telegram
if: success()
run: |
  MESSAGE=" *Imagen Docker subida exitosamente* *Imagen:*
    ${ env.DOCKER_IMAGE_NAME }:${ env.DOCKER_TAG }%OA
    *Docker Hub:* https://hub.docker.com/r/${ secrets.DOCKER_USERNAME }/api-crud-robert%OA *Commit:* ${
    github.sha }%OA *Autor:* ${ github.actor }%OA *
    Rama:* ${ github.ref_name }"
  curl -s -X POST "https://api.telegram.org/bot${ secrets.TELEGRAM_BOT_TOKEN }/sendMessage" \
    -d chat_id="${ secrets.TELEGRAM_CHAT_ID }" \
    -d text="$MESSAGE" \
    -d parse_mode="Markdown"
```

Este step se ejecuta solo si todos los steps anteriores fueron exitosos (if: success()). El mensaje incluye:

- Nombre y etiqueta de la imagen
- Enlace a Docker Hub
- Hash del commit
- Autor del commit
- Rama desde la que se hizo el push

3.3.2. Notificación de error

Listing 10: Notificación cuando ocurre un error

```
- name: Enviar notificacion de error a Telegram
  if: failure()
  run: |
    MESSAGE=" *Error al subir imagen Docker* *Imagen:* ${
      env.DOCKER_IMAGE_NAME }}:${{ env.DOCKER_TAG }}%OA *
      Commit:* ${ github.sha }}%OA *Autor:* ${ github.
      actor }}%OA *Rama:* ${ github.ref_name }}%OA%OA
      Revisa los logs del workflow"
    curl -s -X POST "https://api.telegram.org/bot${ secrets
      .TELEGRAM_BOT_TOKEN }}/sendMessage" \
      -d chat_id="${{ secrets.TELEGRAM_CHAT_ID }}" \
      -d text="$MESSAGE" \
      -d parse_mode="Markdown"
```

Este step se ejecuta solo si algún step anterior falló (`if: failure()`), alertando al equipo sobre el problema.

4. Configuración de Secrets

4.1. Secrets requeridos en GitHub

Para que el workflow funcione correctamente, es necesario configurar los siguientes secrets en el repositorio:

Secret	Descripción
DOCKER_USERNAME	Usuario de Docker Hub
DOCKER_PASSWORD	Token de acceso o contraseña de Docker Hub
TELEGRAM_BOT_TOKEN	Token del bot de Telegram (obtenido de @BotFather)
TELEGRAM_CHAT_ID	ID del chat o canal donde se enviarán las notificaciones

Cuadro 1: Secrets necesarios para el workflow

4.2. Cómo configurar los secrets

1. Ir al repositorio en GitHub
2. Navegar a **Settings** → **Secrets and variables** → **Actions**
3. Click en **New repository secret**
4. Agregar cada secret con su nombre y valor correspondiente

5. Flujo de Trabajo Completo

5.1. Proceso automatizado

1. El desarrollador realiza cambios en el código
2. Se hace commit y push a la rama `main` o `master`
3. GitHub Actions detecta el push (si afecta archivos relevantes)
4. Se ejecuta el workflow:
 - Se descarga el código
 - Se configura Docker Buildx
 - Se autentica en Docker Hub
 - Se construye la imagen Docker
 - Se sube la imagen a Docker Hub
 - Se muestra información en los logs
5. Si todo es exitoso:
 - Se envía notificación de éxito a Telegram
6. Si ocurre un error:
 - Se envía notificación de error a Telegram

6. Ventajas del Sistema

6.1. Automatización

- Eliminación de pasos manuales propensos a errores
- Ahorro de tiempo en el proceso de despliegue
- Construcción consistente de imágenes Docker

6.2. Trazabilidad

- Historial completo de builds en GitHub Actions
- Información detallada del commit asociado a cada imagen
- Identificación rápida del autor y rama

6.3. Notificaciones en tiempo real

- Alertas inmediatas en Telegram sobre el estado del despliegue
- Información contextual del build
- Notificaciones tanto de éxito como de error

6.4. Optimización

- Uso de caché para acelerar construcciones
- Ejecución solo cuando hay cambios relevantes
- Construcción paralela con Docker Buildx

7. Conclusiones

La implementación de este pipeline CI/CD con GitHub Actions proporciona un flujo de trabajo automatizado, eficiente y confiable para el despliegue de la aplicación. La integración con Telegram permite al equipo mantenerse informado en tiempo real sobre el estado de los despliegues, facilitando una respuesta rápida ante cualquier problema.